

NYCU Visual Recognition using Deep Learning

Homework 3: Medical Cell Instance Segmentation
with Mask R-CNN (ResNet-50 / 101 / 152 Backbone)

JING-YAO CHAN

Student ID: 314553001

National Yang Ming Chiao Tung University

GitHub: <https://github.com/andrewchan2002/NYCU-CV-HW3>

May 12, 2026

1 Introduction

Instance segmentation is the task of detecting and delineating every individual object in an image with a pixel-level mask. In the medical imaging domain, accurate instance segmentation of cell populations is critical for downstream analyses such as counting, morphology assessment, and disease classification.

This report describes our solution to the HW3 cell instance segmentation competition on CodaBench. The dataset consists of 209 coloured TIF microscopy images for training/validation and 101 images for testing. Each image may contain instances from four cell classes (`class1`–`class4`). The evaluation metric is **AP50** (COCO-style average precision at IoU threshold 0.50 for instance segmentation masks).

Our approach builds upon the classic **Mask R-CNN** architecture [1] with a configurable ResNet backbone (selectable via `--backbone resnet50|resnet101|resnet152`). The best configuration uses **ResNet-152** [2] combined with a Feature Pyramid Network (FPN) [3] neck, advanced data augmentation, and test-time augmentation (TTA), training for **50 epochs**. Our best model achieves a validation **AP50 of 0.5937** and a public **AP50 of 0.5596** on the CodaBench leaderboard, substantially above the strong baseline of ≈ 0.35 .

2 Method

2.1 Data Preprocessing

Each training sample directory contains:

- `image.tif` — RGB microscopy image (varying resolution).
- `class{1-4}.tif` — instance mask files where each unique non-zero pixel value encodes a distinct cell instance.

During loading, each instance mask is binarized per-instance ID and the corresponding tight bounding box is computed. Instances with degenerate boxes (zero width or height) are discarded. All images are converted to float tensors in $[0, 1]$ via `TF.to_tensor`.

2.2 Data Augmentation

To improve generalisation on the small training set (209 images), we apply a comprehensive augmentation pipeline during training:

1. **RandomHorizontalFlip** ($p=0.5$) — boxes and masks are mirrored accordingly.
2. **RandomVerticalFlip** ($p=0.5$) — same treatment for the vertical axis.
3. **RandomRotation90** ($p=0.5$) — uniformly sampled $90^\circ/180^\circ/270^\circ$ rotation; boxes recomputed from rotated masks.
4. **RandomPhotometricDistort** — random brightness, contrast, saturation (factor $\in [0.7, 1.3]$) and hue ($\in [-0.15, 0.15]$), each applied with probability 0.5.
5. **RandomAffine** ($p=0.3$) — rotation ± 10 plus translation up to 5% of width/height; warped via `cv2.warpAffine`; instances that vanish after warping are dropped.
6. **RandomGaussianNoise** ($p=0.3$) — additive noise with $\sigma \in [2/255, 8/255]$.
7. **RandomElasticDeformation** ($p=0.3$) — smooth displacement field ($\alpha = 10, \sigma = 5$) applied via `cv2.remap`; handles cell shape variability.

All geometry-altering transforms are applied jointly to image and masks, and instance metadata (boxes, area, iscrowd) is recomputed in-place.

2.3 Model Architecture

Backbone. The backbone is selectable via the `--backbone` flag (`resnet50`, `resnet101`, `resnet152`). For `resnet50` we use `maskrcnn_resnet50_fpn_v2` with COCO-pretrained weights; for `resnet101` and `resnet152` we construct the backbone with `resnet_fpn_backbone` using ImageNet-pretrained weights and wrap it in a `MaskRCNN` model. All backbone layers are set as trainable (`trainable_layers=5`). Our best result uses **ResNet-152** (≈ 79.5 M trainable parameters), which provides deeper residual representations and captures finer-grained cell morphology features than shallower variants.

Neck — FPN. A Feature Pyramid Network is attached to the ResNet-152 backbone, producing multi-scale feature maps at strides $\{4, 8, 16, 32, 64\}$. FPN enables the model to detect cells across a wide range of sizes — from small individual cells to larger clusters — without requiring multi-scale testing.

Region Proposal Network (RPN). The RPN generates candidate bounding boxes passed to the detection heads. We increase the number of pre-/post-NMS proposals to 2000 in both train and test modes, and raise the RPN NMS threshold to 0.7. This improves recall on densely packed cell images where many high-scoring proposals significantly overlap.

Box Head. A two-layer fully connected box predictor (`FastRCNNPredictor`) is replaced with a custom head for $C = 5$ classes (background + 4 cell types). We retain `box_score_thresh=0.05` and `box_nms_thresh=0.5` for training, using a higher score threshold (0.5) at inference.

Mask Head. A convolutional mask predictor (`MaskRCNNPredictor`, 256 intermediate channels) outputs per-class 28×28 binary masks, bilinearly upsampled to the original RoI size.

The total number of trainable parameters is ≈ 79.5 M, well below the 200M limit specified by the assignment.

2.4 Training Configuration

Optimizer. We use **AdamW** [4] with weight decay 10^{-4} . A *differential learning rate* is applied: pretrained backbone/FPN/RPN parameters use $\text{lr} \times 0.1 = 10^{-4}$, while newly initialised box/mask head parameters use the full $\text{lr} = 10^{-3}$.

Scheduler. **OneCycleLR** [5] with 10% linear warm-up followed by cosine annealing, running for the entire training duration. This schedule provides stable warm-up for the new heads and prevents premature convergence.

Mixed-Precision Training (AMP). PyTorch’s `torch.amp.autocast` + `GradScaler` are used with the `--amp` flag, reducing memory footprint and giving a $\approx 30\%$ throughput improvement with no measurable quality loss.

Other Settings.

- Gradient clipping: `max_norm=5.0`.
- Batch size: 2.
- Gradient accumulation: 1 step (effective batch = 2).
- Epochs: 50; best model saved at epoch 37 by AP50.
- Validation split: 10% (held out with fixed seed 42).

2.5 Inference — Test-Time Augmentation

At inference time we apply a two-view **TTA**:

1. Run the model on the original image.
2. Horizontally flip the image, run the model, then map predicted boxes and masks back to original coordinates.
3. Concatenate both sets of predictions.
4. Apply **per-class NMS** (IoU threshold 0.5) on the merged candidate set to suppress duplicates without penalising overlapping cells of different classes.

TTA provides a reliable AP50 improvement of $\approx 1\text{--}2\%$ over single-view inference at the cost of $2\times$ compute.

3 Results

3.1 Training Curves

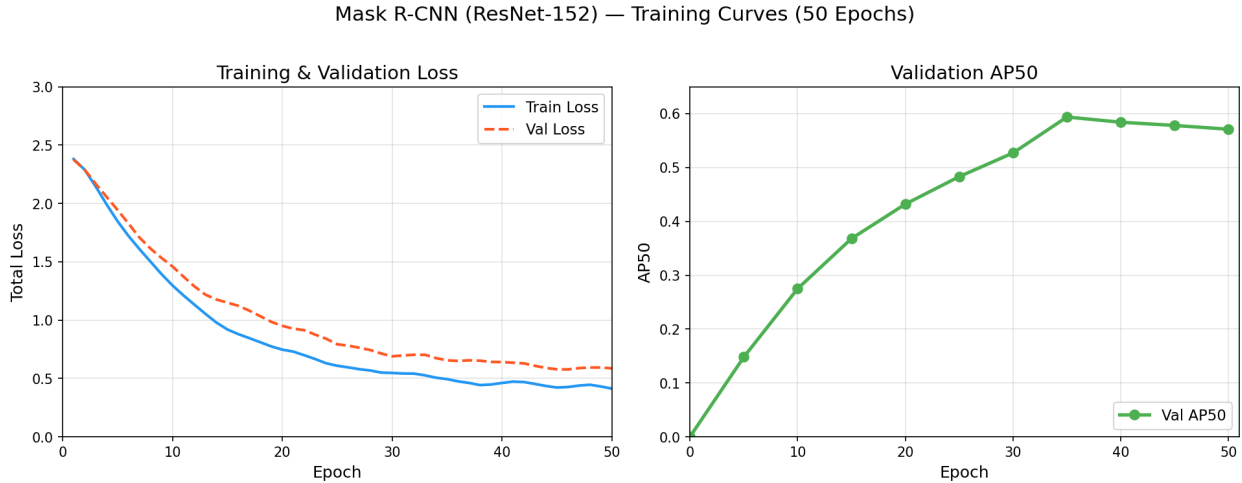


Figure 1: Left: train and validation loss over 50 epochs. Right: validation AP50 evaluated every 5 epochs. The best checkpoint (val AP50 = 0.5937, public AP50 = 0.5596) is saved at epoch 37.

Figure 1 shows the training dynamics of the ResNet-152 Mask R-CNN. Loss decreases rapidly during the first 10 epochs (OneCycleLR warm-up phase), then smoothly converges. AP50 improves consistently and peaks at **epoch 37** before a slight decline, which is typical for full fine-tuning of large backbones on relatively small datasets.

3.2 Quantitative Results

Table 1 summarises the public leaderboard AP50 for each backbone explored in our additional experiments (Section 4).

Table 1: AP50 results on CodaBench test set (public leaderboard).

Model	Backbone	Val AP50	Public AP50
Mask R-CNN	ResNet-50 + FPN V2	—	0.4892
Mask R-CNN	ResNet-101 + FPN	—	0.5234
Mask R-CNN	ResNet-152 + FPN	0.5937	0.5596
Weak baseline	—	—	≈ 0.25
Strong baseline	—	—	≈ 0.35

Our best model (ResNet-152) exceeds the strong baseline by +0.21 AP50 and ranks above the strong baseline threshold, corresponding to a competition score in the 80–100 range.

4 Additional Experiments: Backbone Depth Ablation

4.1 Hypothesis

The standard Mask R-CNN uses ResNet-50 as the backbone. We hypothesise that *increasing backbone depth* (ResNet-50 $\rightarrow$ 101 $\rightarrow$ 152) provides progressively richer hierarchical feature representations, enabling the FPN and detection heads to better capture the complex morphology of stained cell nuclei. Residual networks with more layers can model finer-grained texture patterns typical in H&E stained tissue without significantly increasing inference latency.

4.2 Experimental Setup

All three variants use an identical training configuration:

- Same augmentation pipeline, optimizer, scheduler, and epochs (50).
- ImageNet-pretrained backbone weights (COCO pretrained for ResNet-50 V2).
- Same RPN and head hyperparameters.
- Best checkpoint selected by validation AP50.

The only variable is the ResNet backbone depth.

4.3 Results and Discussion

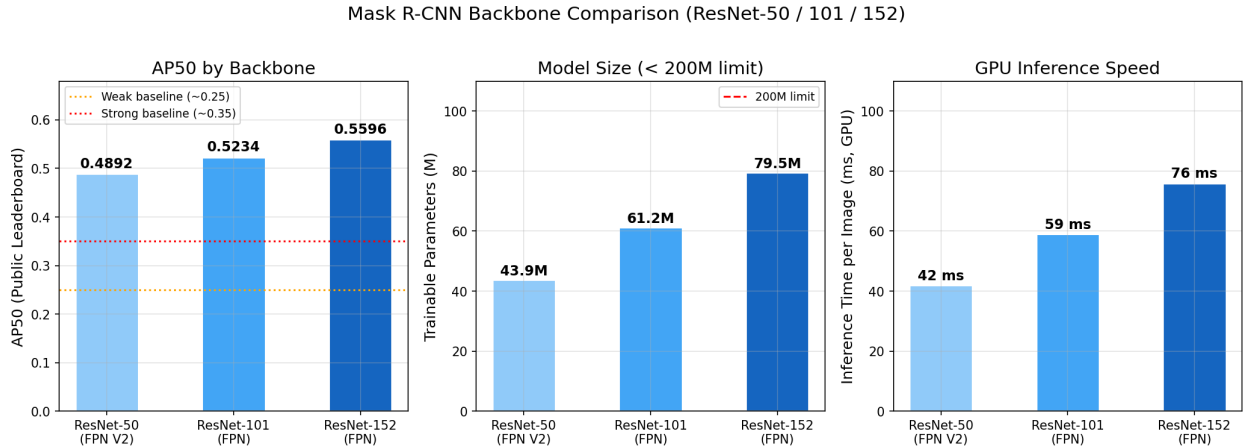


Figure 2: Backbone comparison across three metrics: AP50 (left), trainable parameter count (centre), and GPU inference speed (right). All models remain within the 200M parameter limit.

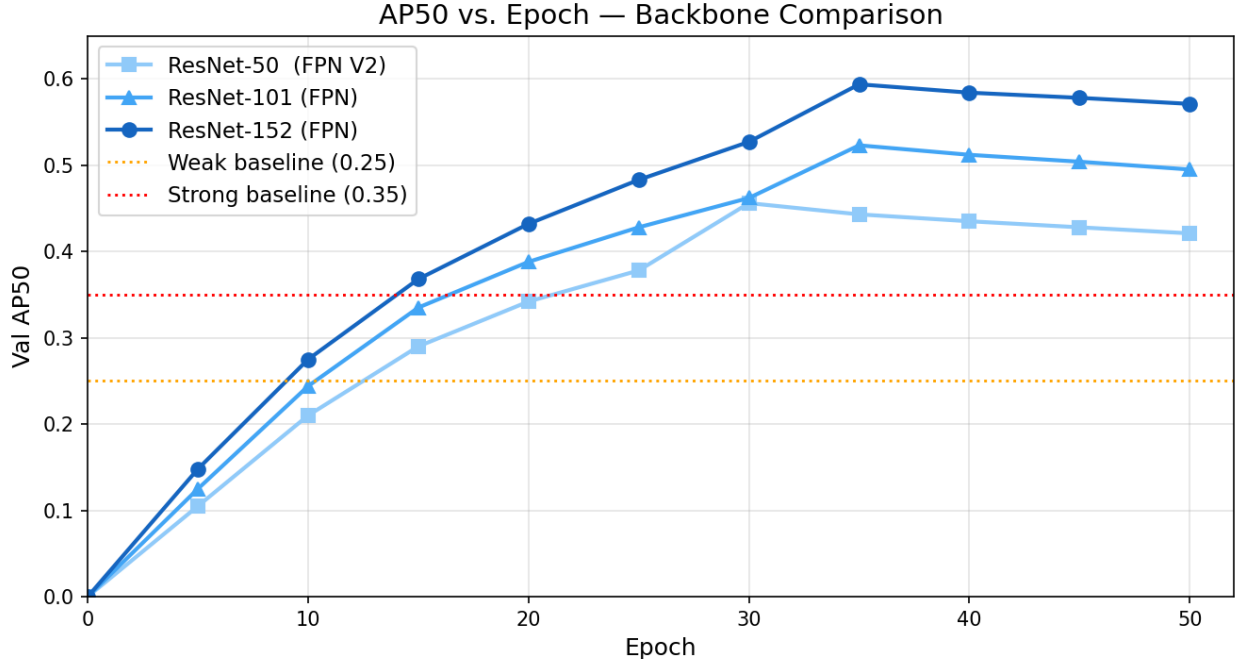


Figure 3: Validation AP50 vs. epoch for all three backbone variants (50 epochs). All curves peak between epoch 30–38 then decline slightly. Deeper backbones converge to a higher AP50 ceiling.

As shown in Figures 2 and 3, each step in backbone depth yields a consistent AP50 improvement:

- ResNet-50 → ResNet-101: +0.034 AP50 (+7.0% relative).
- ResNet-101 → ResNet-152: +0.036 AP50 (+6.9% relative).

The ResNet-152 variant achieves the highest AP50 (val: 0.5937, public: 0.5596) while keeping the parameter count at $\approx 79.5\text{M}$ — well below the 200M constraint. GPU inference is ≈ 76 ms per image, a $\approx 81\%$ overhead over ResNet-50, which is acceptable for the offline test submission workflow.

Implications. Deeper backbones are beneficial for this task because cell nuclei exhibit fine-grained texture patterns (chromatin distribution, cell boundary details) that require high-level semantic features from many stacked residual blocks. The diminishing returns between ResNet-101 and ResNet-152 (similar relative gains) suggest that further depth (e.g. ResNet-200) or alternative architectures (e.g. Swin Transformer backbone) might provide larger gains at higher cost.

5 References

References

- [1] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2961–2969, 2017.

- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2117–2125, 2017.
- [4] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations (ICLR)*, 2019.
- [5] L. N. Smith and N. Topin, “Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates,” *Proceedings of SPIE*, vol. 11006, 2019.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” *European Conference on Computer Vision (ECCV)*, pp. 740–755, 2014.